

HTTP-сервер

Конспект: сети, протокол HTTP, простой HTTP-сервер на Python

1. Как работает сеть. Протоколы и модель OSI

Компьютерная сеть передаёт данные от одного устройства к другому. Чтобы разные производители оборудования и программ могли «договориться» между собой, придумали строгие правила — **протоколы**.

OSI (Open System Interconnection) — эталонная модель взаимодействия открытых систем, разработана ISO (International Standards Organization) в 1984 году. Она описывает передачу данных как прохождение через 7 уровней, каждый из которых решает свою задачу.

Главный принцип: данные разбиваются на **пакеты** — единицы информации, передаваемые как единое целое. На каждом уровне отправителя к пакету добавляется служебная информация (адрес, проверка ошибок и т.д.), а на стороне получателя эта информация снимается в обратном порядке — слой за слоем, пока данные не примут исходный вид.

7 уровней модели OSI

Уровень	Название	Что делает
7	Прикладной (Application)	Интерфейс для программ пользователя (браузер, почта, файлообмен)
6	Представительский (Presentation)	«Переводчик» — приводит данные к общему формату, шифрует, сжимает
5	Сеансовый (Session)	Устанавливает и завершает сеанс связи между приложениями
4	Транспортный (Transport)	Гарантирует доставку пакетов без ошибок и в правильном порядке
3	Сетевой (Network)	Определяет маршрут от отправителя к получателю (адресация)
2	Канальный (Data Link)	Упаковывает данные в кадры (frames), проверяет ошибки передачи
1	Физический (Physical)	Передача «сырых» битов по кабелю, сигналы

Просто для понимания: чем выше уровень — тем сложнее и «ближе к пользователю» задача. Нижние уровни (1–2) — это провода и сигналы, верхние (5–7) — то, с чем работают приложения.

2. Модель TCP/IP

TCP/IP (Transmission Control Protocol / Internet Protocol) — более практичная модель, на которой реально работает интернет. Появилась в 1972–83 гг. У неё 4 уровня (упрощённый вариант OSI):

Уровень TCP/IP	Функция
Канальный (Network Access)	Работа с оборудованием — Ethernet, Wi-Fi
Межсетевой (Internet)	Маршрутизация по IP-адресам (IPv4/IPv6)
Транспортный (Transport)	Надёжная доставка данных — здесь работают TCP и UDP
Прикладной (Application)	Протоколы конкретных сервисов — HTTPS, SMTP, DHCP

Порты и сокеты

- **Порт** — номер, определяющий, какому именно приложению на устройстве адресованы данные.

- **Сокет** — это IP-адрес + порт вместе. Например, `195.34.32.116:53` — конкретный компьютер и конкретный сервис на нём (53 — обычно DNS).
- Стандартные порты: 80 — HTTP, 443 — HTTPS, 21 — FTP.

IP, TCP и UDP — в чём разница

- **IP (Internet Protocol)** — просто доставляет пакеты по адресу. Не проверяет ошибки, не гарантирует порядок.
- **TCP (Transmission Control Protocol)** — устанавливает соединение, гарантирует, что все данные дойдут в правильном порядке без ошибок (за счёт подтверждений и повторной отправки). Медленнее, но надёжнее.
- **UDP (User Datagram Protocol)** — не устанавливает соединение, ничего не гарантирует, зато быстрый. Используется там, где скорость важнее надёжности (видеостриминг, онлайн-игры).

IP — это «адрес доставки», а TCP/UDP — это «как именно везём»: TCP — аккуратно и с подтверждением, UDP — быстро и без гарантий.

3. IP-адреса и маски подсети

IP-адрес — уникальный идентификатор устройства в сети.

- **IPv4** — 32 бита, записывается как 4 числа 0–255 через точку (`192.168.1.1`). Всего ~4,3 млрд адресов.
- **IPv6** — 128 бит, записывается как 8 групп шестнадцатеричных чисел (создан из-за нехватки IPv4-адресов).

Маска подсети

Адрес делится на две части: **сеть** и **хост** (конкретное устройство). Маска показывает, где заканчивается «сеть» и начинается «хост»: единицы — часть сети, нули — часть хоста.

Чаще используется **CIDR-нотация (Classless Inter-Domain Routing)** — короткая запись маски через «/». Например, `192.168.1.1/24` означает, что первые 24 бита — это адрес сети.

Как посчитать адрес сети (пример для `192.168.1.1/24`):

1. Переводим IP и маску в двоичный вид.
2. Применяем побитовую операцию **И (AND)** между IP и маской.
3. Результат — это адрес сети: `192.168.1.0`.

Широковещательный адрес (broadcast) — адрес, где все биты хоста равны 1. Для сети `192.168.1.0/24` это `192.168.1.255` — сообщение на этот адрес получают все устройства подсети.

Поэтому в подсети `/24` реально устройствам доступны адреса `192.168.1.1 – 192.168.1.254` (`.0` — адрес сети, `.255` — broadcast).

4. Приватные IP-адреса и NAT

RFC 1918 (Request for Comments 1918) — документ, описывающий диапазоны адресов только для локальных сетей (не маршрутизируются в интернете):

Диапазон	Маска	Кол-во адресов
10.0.0.0 – 10.255.255.255	/8	16 777 216
172.16.0.0 – 172.31.255.255	/12	1 048 576
192.168.0.0 – 192.168.255.255	/16	65 536

NAT (Network Address Translation) — механизм, позволяющий нескольким устройствам с приватными адресами выходить в интернет через один публичный IP-адрес роутера. Именно поэтому у вас дома все устройства

имеют адреса вида `192.168.x.x`, но в интернет выходят «под одним лицом».

5. Как устроен интернет. Протокол HTTP

- **Хостинг** — услуга размещения сайта (файлов + программ для их обработки) на сервере, постоянно подключённом к сети.
- **DNS-сервер (Domain Name System)** — переводит доменное имя (`google.com`) в IP-адрес.

HTTP (HyperText Transfer Protocol)

Основной протокол передачи данных в интернете. Работает по принципу «запрос — ответ»: клиент (браузер) отправляет запрос, сервер отвечает.

Две ключевые особенности:

- **Синхронный** — пока не пришёл ответ на запрос, следующий запрос не отправить.
- **Без сохранения состояния (stateless)** — сервер не помнит, что было в предыдущем запросе. Для «запоминания» пользователя используют cookie или сессии.

Этапы запроса: DNS-запрос (домен → IP) → установка соединения с сервером → отправка данных → ожидание ответа → получение данных.

URI и URL — в чём разница

- **URI (Uniform Resource Identifier)** — идентификатор ресурса вообще (как имя).
- **URL (Uniform Resource Locator)** — конкретный вид идентификатора, который ещё и говорит, **как** получить доступ к ресурсу (протокол + адрес).

Проще: URI — это «что», URL — это «что и как найти». Любой URL — это URI, но не наоборот.

Структура URL

```
схема://[логин[:пароль]@]хост[:порт][[/путь]][?запрос][#фрагмент]
```

Пример: `http://user:password@host.com:80/resource/path/?query=name&ttt=123#hash`

- **схема** — протокол (http, https, ftp)
- **хост** — доменное имя или IP
- **порт** — порт подключения (по умолчанию 80 для http, 443 для https — браузер подставляет их сам)
- **путь** — расположение ресурса
- **запрос (query)** — параметры после «?», разделены «&»
- **фрагмент (hash/якорь)** — часть после «#», указывает на конкретное место на странице

6. Структура HTTP-сообщения

Сообщение состоит из **стартовой строки**, **заголовков (headers)**, пустой строки и (опционально) **тела запроса**.

HTTP-методы

Метод	Назначение
GET	Получить данные (не должен менять состояние ресурса)
POST	Создать новый ресурс, данные передаются в теле запроса
PUT	Обновить/создать ресурс. Идемпотентен — повтор даёт тот же результат
DELETE	Удалить ресурс
HEAD	Как GET, но только заголовки, без тела
PATCH	Частичное обновление ресурса
OPTIONS	Узнать, какие методы поддерживает сервер

HTML-формы поддерживают в браузере нативно только GET и POST (историческое ограничение). Остальные методы (PUT , PATCH , DELETE) доступны через JavaScript.

Пример запроса

```
GET /index.html HTTP/1.1
Host: www.example.com
User-Agent: Mozilla/5.0
Accept: text/html
Connection: keep-alive
```

Разбор: метод GET , ресурс /index.html , версия протокола HTTP/1.1 ; дальше — заголовки с доп. информацией (хост, браузер, тип соединения).

Пример ответа

```
HTTP/1.1 200 Ok
Server: nginx
Content-Type: text/html; charset=UTF-8
Cache-Control: no-cache
```

Разбор: версия протокола + код статуса (200 Ok — успех), тип сервера, тип контента, настройки кеширования.

7. HTTP-заголовки

Заголовки передают дополнительную информацию о запросе, ответе или самих данных. Делятся на 4 категории:

Категория	Где используется	Примеры
Общие (General)	И в запросах, и в ответах	Cache-Control , Connection , Date
Запроса (Request)	Только в запросах	Accept , Host , User-Agent
Ответа (Response)	Только в ответах	Server , WWW-Authenticate
Тела (Entity)	Описывают тело сообщения	Content-Type , Content-Length , Content-Encoding

Ключевые заголовки

Заголовок	Назначение	Пример
Content-Type	MIME-тип содержимого тела (как интерпретировать данные)	Content-Type: application/json
Host	Доменное имя сервера — важно, когда один сервер держит несколько сайтов	Host: www.example.com
User-Agent	Информация о браузере/клиенте, который отправил запрос	User-Agent: Mozilla/5.0 ...
Accept	Какие типы контента клиент готов принять	Accept: text/plain
Authorization	Данные для аутентификации (часто закодированы)	Authorization: Basic YWxhZGRpbjpvGVu...
Cache-Control	Правила кеширования ответа	Cache-Control: max-age=3600
Connection	Держать соединение открытым или закрыть после ответа	Connection: keep-alive

Пример Authorization: строка после «Basic» — это **Base64** (способ кодирования данных в текст) от связки "логин:пароль". Это не шифрование, а просто кодировка, поэтому Basic-авторизацию используют только вместе с HTTPS.

8. Коды состояния ответов сервера

Трёхзначное число в начале ответа сервера. Первая цифра определяет группу (класс) ответа.

Группа	Смысл	Примеры
1xx	Информационные	100 Continue — сервер принял начало запроса, можно продолжать отправку
2xx	Успешные	200 OK — успех; 201 Created — ресурс создан; 204 No Content — успех без тела ответа
3xx	Перенаправления	301 Moved Permanently — ресурс переехал навсегда; 302 Found — переехал временно; 304 Not Modified — используй кеш
4xx	Ошибки клиента	400 Bad Request — неверный синтаксис запроса; 401 Unauthorized — нужна аутентификация; 403 Forbidden — доступ запрещён; 404 Not Found — не найдено
5xx	Ошибки сервера	500 Internal Server Error — общая ошибка сервера; 503 Service Unavailable — сервер временно недоступен

9. Простой HTTP-сервер на Python

В Python есть встроенный модуль `http.server`, который реализует простой HTTP-сервер. Для «боевых» проектов не подходит, но отлично годится для обучения.

```
from http.server import HTTPServer, BaseHTTPRequestHandler

class SimpleHTTPRequestHandler(BaseHTTPRequestHandler):

    def do_GET(self):
        self.send_response(200)
        self.end_headers()
        self.wfile.write(b'Hello, world!')

    def do_POST(self):
        pass
```

Сервер отвечает на GET-запрос строкой 'Hello, world!', а на POST-запрос ничего не делает.

Клиентская часть

```
from http import client

h1 = client.HTTPConnection('localhost', 8001)
h1.request("GET", "/")

res = h1.getresponse()
print(res.status, res.reason)

data = res.read()
print(data)
```

Класс `HTTPConnection` подключается к хосту и порту сервера, метод `request` отправляет GET-запрос, а `getresponse` возвращает ответ — можно проверить статус (`res.status`), причину (`res.reason`) и прочитать тело (`res.read()`).

Запуск сервера в отдельном потоке

Чтобы сервер не блокировал выполнение остального кода, его запускают в отдельном потоке (`Thread`):

```
from threading import Thread
from time import sleep
from http import client
from http.server import HTTPServer, BaseHTTPRequestHandler

class SimpleHTTPRequestHandler(BaseHTTPRequestHandler):

    def do_GET(self):
        self.send_response(200)
        self.end_headers()
        self.wfile.write(b'Hello, world!')

    def do_POST(self):
        pass

httpd = HTTPServer(('localhost', 8001), SimpleHTTPRequestHandler)
server = Thread(target=httpd.serve_forever)
server.start()
sleep(0.5)

h1 = client.HTTPConnection('localhost', 8001)
h1.request("GET", "/")

res = h1.getresponse()
print(res.status, res.reason)
data = res.read()
print(data)

httpd.shutdown()
```

Порт `8001` указан явно. Если порт не указать, по умолчанию используется `80`, а он обычно требует прав администратора.

10. Построение HTTP-сервера: маршрутизация

Идём дальше и строим простой сайт из двух страниц: `index.html` (главная) и `contact.html` (форма обратной связи). Для стилизации используется Bootstrap — сама стилизация в этой теме не разбирается, фокус на логике сервера.

Определяем маршруты

- `'/'` — главная страница;
- `'/contact'` — страница с формой отправки сообщения.

В HTML-коде ссылки меняются на эти маршруты:

```
<ul class="navbar-nav">
  <li class="nav-item">
    <a class="nav-link" href="/">Home</a>
  </li>
  <li class="nav-item">
    <a class="nav-link" href="/contact">Write to me</a>
  </li>
</ul>
```

Эти маршруты попадают в переменную `self.path` класса-обработчика.

Файл `main.py`

```
from http.server import HTTPServer, BaseHTTPRequestHandler
import urllib.parse

class HttpHandler(BaseHTTPRequestHandler):
    def do_GET(self):
        pr_url = urllib.parse.urlparse(self.path)
        if pr_url.path == '/':
            self.send_html_file('index.html')
        elif pr_url.path == '/contact':
            self.send_html_file('contact.html')
        else:
            self.send_html_file('error.html', 404)

    def send_html_file(self, filename, status=200):
        self.send_response(status)
        self.send_header('Content-type', 'text/html')
        self.end_headers()
        with open(filename, 'rb') as fd:
            self.wfile.write(fd.read())

def run(server_class=HTTPServer, handler_class=HttpHandler):
    server_address = ('', 8000)
    http = server_class(server_address, handler_class)
    try:
        http.serve_forever()
    except KeyboardInterrupt:
        http.server_close()

if __name__ == '__main__':
    run()
```

Разбор ключевых моментов

`urllib.parse.urlparse(self.path)` — правильный способ разбора адреса. Если просто читать `self.path`, при запросе `/contact?name=Test` в нём окажется вся строка целиком: `/contact?name=Test`. А `urlparse` аккуратно

разделяет её на части: `path='/contact'` и `query='name=Test'` — так параметры запроса (query) не мешают определению маршрута.

`send_html_file(filename, status=200)` — метод, который читает файл и отправляет его браузеру: код статуса → заголовок `Content-type` → пустая строка (`end_headers()`) → содержимое файла.

Сама маршрутизация — обычная цепочка `if / elif / else`: если маршрут не совпал ни с одним известным — возвращаем страницу `404 Not Found`.

Если запустить сервер и открыть `http://localhost:8000/`, маршруты уже работают, но пропадают картинки и стили из `style.css` — фона нет. Такие файлы (не HTML) называют **статическими ресурсами**. Их подключение — тема следующего раздела.

11. Статические ресурсы

В логах сервера видно: при запросе главной страницы браузер сразу же просит ещё файл стилей и картинку — и получает `404`, потому что приложение ничего не знает об этих файлах.

```
127.0.0.1 - - [16/Sep/2022 18:20:06] "GET / HTTP/1.1" 200 -
127.0.0.1 - - [16/Sep/2022 18:20:07] "GET /style.css HTTP/1.1" 404 -
127.0.0.1 - - [16/Sep/2022 18:20:07] "GET /images/gupalo.jpg HTTP/1.1" 404 -
```

Браузер находит ссылки на эти файлы прямо в `index.html` и запрашивает их отдельно:

```
<link rel="stylesheet" href="style.css" />
...

```

Решение простое: проверять, существует ли такой файл на сервере, и если да — отправлять его браузеру. Но нужно ещё правильно указать **тип содержимого**, иначе браузер не поймёт, как его показывать.

MIME-типы

MIME (Multipurpose Internet Mail Extensions) — стандарт обозначения типа передаваемых данных. Для HTML мы уже отправляли заголовок `Content-type: text/html` — это и есть MIME-тип. Для картинок, стилей и других файлов тип нужно определять отдельно. В Python для этого есть готовый модуль `mimetypes`.

```
def send_static(self):
    self.send_response(200)
    mt = mimetypes.guess_type(self.path)
    if mt:
        self.send_header("Content-type", mt[0])
    else:
        self.send_header("Content-type", 'text/plain')
    self.end_headers()
    with open(f'{self.path}', 'rb') as file:
        self.wfile.write(file.read())
```

Если `mimetypes.guess_type` определил тип — отправляем его. Если не смог — считаем файл обычным текстом (`text/plain`).

Подключаем к маршрутизации

```

if pr_url.path == '/':
    self.send_html_file('index.html')
elif pr_url.path == '/contact':
    self.send_html_file('contact.html')
else:
    if pathlib.Path().joinpath(pr_url.path[1:]).exists():
        self.send_static()
    else:
        self.send_html_file('error.html', 404)

```

Логика: если путь не совпал с известными страницами — проверяем, существует ли такой файл на диске. Если да — отдаём как статический ресурс, если нет — возвращаем страницу `404`.

После этого в логах порядок: все три запроса возвращают `200`, и на странице появляются и картинка, и стили.

12. Обработка формы (POST-запрос)

Приём данных из формы обрабатывается методом `do_POST`. Так как в приложении форма всего одна, маршрутизация внутри `do_POST` не нужна — `self.path` всегда будет `/contact` (это значение атрибута `action` формы).

```

<form action="/contact" method="POST" enctype="application/x-www-form-urlencoded">
  <input name="username" type="text" />
  <input name="email" type="email" />
  <textarea name="message"></textarea>
  <button type="submit">Send</button>
</form>

```

Код обработчика

```

def do_POST(self):
    data = self.rfile.read(int(self.headers['Content-Length']))
    data_parse = urllib.parse.unquote_plus(data.decode())
    data_dict = {key: value for key, value in [el.split('=') for el in data_parse.split('&')]}
    self.send_response(302)
    self.send_header('Location', '/')
    self.end_headers()

```

Разбор по шагам

1. Чтение данных. Браузер собирает пары `имя_поля=значение` из всех полей формы (по атрибутам `name`), соединяет их знаком `&` и отправляет в теле запроса. Размер этих данных в байтах передаётся в заголовке `Content-Length` — по нему сервер понимает, сколько байт нужно прочитать:

```

data = self.rfile.read(int(self.headers['Content-Length']))
# b'username=krabaton&email=krabat%40test.com&message=Hello+my+friend'

```

2. Декодирование. При `enctype="application/x-www-form-urlencoded"` браузер заменяет пробелы на `+` и кодирует спецсимволы (например, `@` становится `%40`). Чтобы вернуть строку в читаемый вид, применяем `urllib.parse.unquote_plus`:

```

data_parse = urllib.parse.unquote_plus(data.decode())
# username=krabaton&email=krabat@test.com&message=Hello my friend

```

3. Превращение в словарь. Строку разбиваем по `&`, затем каждую пару — по `=`:

```
data_dict = {key: value for key, value in [el.split('=') for el in data_parse.split('&')]}
# {'username': 'krabaton', 'email': 'krabat@test.com', 'message': 'Hello my friend'}
```

Такой словарь уже можно сохранить, например, в базу данных.

4. Редирект. После обработки формы сервер отвечает статусом `302` (временное перенаправление) и заголовком `Location: /` — браузер получает эту пару и сам переходит на главную страницу.

На этом простое веб-приложение на `http.server` завершено: есть маршрутизация, статические ресурсы и обработка формы через POST.

13. Шаблонизатор Jinja

Писать HTML-документы «с нуля» в Python приходится редко. Чаще нужно взять готовый HTML-шаблон и подставить в него данные — для этого удобно использовать пакет **Jinja**.

```
pip install jinja2
```

У Jinja есть собственный простой язык разметки внутри HTML-текста. Он не привязан именно к HTML — можно рендерить любой текст, куда встраивается Python-код, циклы и условия.

- `{{ }}` — сюда подставляется значение переменной (Python-выражение, результат превращается в строку и вставляется в текст).
- `{% %}` — управляющие конструкции: `for`, `if`. Всё между `{% for %}` и `{% endfor %}` (или `{% if %}` и `{% endif %}`) повторяется или выполняется по правилам цикла/условия.

Простой пример

```
from jinja2 import Template

name = 'Bill'
age = 28

tm = Template("My name is {{ name }} and I am {{ age }}")
msg = tm.render(name=name, age=age)

print(msg) # My name is Bill and I am 28
```

Чтобы получить готовый текст, у шаблона вызывается метод `render()`, куда передаются именованные аргументы. Имена переменных в шаблоне и имена аргументов при вызове `render()` должны совпадать.

Цикл for по списку словарей

```
persons = [
    {'name': 'Andrej', 'age': 34},
    {'name': 'Mark', 'age': 17},
    {'name': 'Thomas', 'age': 44},
]

rows_tmp = Template("""{% for person in persons -%}
    {{ person.name }} {{ person.age }}
{% endfor %}""")

print(rows_tmp.render(persons=persons))
```

Важная деталь: в Jinja обращение через точку (`person.name`) и обращение через квадратные скобки, как к словарю (`person['name']`), внутри шаблона работают одинаково — можно использовать точку, даже если `person` на самом деле словарь.

Шаблоны из файлов

Когда шаблон хранится в отдельном HTML-файле, используется класс `Environment` с загрузчиком `FileSystemLoader` — он указывает папку, где искать шаблоны.

Файл `persons.html` :

```
<ul>
    {% for person in persons -%}
    <li>{{ person.name }} {{ person.age }}</li>
    {% endfor %}
</ul>
```

Скрипт, который загружает шаблон и подставляет данные:

```
from jinja2 import Environment, FileSystemLoader

env = Environment(loader=FileSystemLoader('.'))
template = env.get_template("persons.html")

persons = [{'name': 'Andrej', 'age': 34}]

output = template.render(persons=persons)

with open("new_persons.html", "w", encoding='utf-8') as fh:
    fh.write(output)
```

Результат — готовый HTML-файл, где вместо `{{ }}` и `{% %}` подставлены реальные данные (список `<li>` для каждого человека).

Итог: шаблон один раз пишется как заготовка с «дырками» (`{{ }}` , `{% %}`), а дальше многократно заполняется новыми данными через `render()` . На практике Jinja чаще используется вместе с веб-фреймворками (например, Django), а не напрямую с `http.server` .

14. Построение HTTP-запросов

Отправлять HTTP-запросы из Python можно двумя способами: встроенным модулем `urllib.request` или сторонним пакетом `requests` .

Встроенный urllib.request

Работает без установки, но многое приходится настраивать вручную:

```
import urllib.request

with urllib.request.urlopen('https://www.python.org/') as f:
    print(f.read(300))
```

Умеет всё базовое: задать метод запроса, заголовки, тело запроса, прочитав статус-код ответа.

Пакет requests

Решает типичные проблемы ручной работы с HTTP: сохранение cookie между запросами, переиспользование открытых TCP-соединений (экономит время и ресурсы), обработку таймаутов, повтор запроса при обрыве сети. Самый популярный пакет для HTTP-запросов в Python — благодаря удобному интерфейсу и подробной документации.

```
pip install requests
```

```
import requests

r = requests.get('https://api.github.com/events')
print(r.text[:300])
```

Пакет сам определяет кодировку ответа по заголовкам и сам распаковывает сжатые ответы — без ручной настройки.

Основные приёмы

Задача	Код
Получить и распарсить JSON	<pre>response = requests.get(url) data = response.json()</pre>
GET с параметрами	<pre>requests.get(url, params={'key': 'value'})</pre>
Обработка ошибок 4xx/5xx	<pre>response.raise_for_status()</pre> → выбрасывает <code>HTTPError</code>
Таймаут	<pre>requests.get(url, timeout=2)</pre>
Заголовки (авторизация)	<pre>requests.get(url, headers={'Authorization': 'Bearer TOKEN'})</pre>
POST с JSON в теле	<pre>requests.post(url, json=payload)</pre>
Отключить редиректы	<pre>requests.get(url, allow_redirects=False)</pre>

Обработка ошибок и таймаут

```
import requests

try:
    response = requests.get('https://api.example.com/data', timeout=2)
    response.raise_for_status()
    data = response.json()
    print(data)
except requests.exceptions.Timeout:
    print('The request timed out')
except requests.exceptions.HTTPError as err:
    print(f'HTTP Error: {err}')
```

Сессия (Session)

Если нужно сделать несколько запросов к одному серверу, объект `Session` переиспользует соединение и хранит общие заголовки/cookie между запросами — это быстрее, чем открывать новое соединение на каждый запрос:

```
with requests.Session() as session:
    session.headers.update({'Authorization': 'Bearer YOUR_ACCESS_TOKEN'})
    response = session.get('https://api.example.com/data')
    print(response.json())
```

Практический пример: курс валют ПриватБанка

```
import requests

response = requests.get('https://api.privatbank.ua/p24api/pubinfo?exchange&json&coursid=11')
exchange_rate = response.json()
print(exchange_rate)
```

Ответ — готовый список словарей с курсами валют:

```
[
  {"ccy": "USD", "base_ccy": "UAH", "buy": "36.56860", "sale": "37.45318"},
  {"ccy": "EUR", "base_ccy": "UAH", "buy": "36.43000", "sale": "37.73585"}
]
```

Три строки кода — и данные из внешнего API уже готовы к использованию: `get()` отправил запрос, `.json()` превратил ответ в Python-объект (список словарей).